

In fact there can be use cases where following design patterns automatically achieve certain non functional goals in the requirement.

There are many reasons to use design patterns when applicable in programs, such as: -

- a. Design Patterns give a software developer, an array of tried and tested solutions to common problems, thus reducing the technical risk to the project by not having to deploy a new and untested design.
- b. They are language neutral and thus can be applied to any language that supports object-orientation.
- c. They aid communication by the very fact that they are well documented and can be researched if that is not the case.
- d. They have a proven track record as they are already widely used and thus reduce the technical risk to the project.
- e. They are highly flexible and scalable and can be used in practically any type of domain or application.

Design Patterns, despite their initial learning curve, are a very worthwhile investment. They will enable you to implement tried and tested solutions to problems, thus saving time and effort during the implementation stage of the software development lifecycle. By using well understood and documented solutions, the final product will have a much higher degree of comprehension. If the solution is easier to comprehend, then by extension, it will also be easier to maintain.

These are just some of the many practices and principles for good programming. Writing simple, readable and easily extendable code goes a long way in effectiveness, often as important than the actual solution itself. There is seldom code written by one person. **d**



Code repositories are essential
for version control